Safety-based Termination Analysis With Transition Invariant Generation

Transition Invariant-based
Remove generation!!
Think on the title

Konstantin Britikov Martin Blicha Grigory Fedyukovich Natasha Sharygina

***Abstract*—Termination analysis of infinite-state systems is a fundamental problem in formal methods. Existing techniques for termination and nontermination are usually separated, concentrating on a particular problem. However the two are theoretically interdependent. The core intuition of this paper is that the intermediate artifacts of termination and nontermination analysis can be used to mutually guide each other. We present a new approach to termination analysis based on the interpolation-based construction of well-founded transition invariants. Our approach leverages Craig interpolation to guide transition invariant generation with respect to the conditions relevant to termination. Our technique extends safety-based nontermination analysis, allowing it to prove both termination and nontermination within a unified framework. We implemented our approach in Golem CHC solver and evaluated it on benchmarks from the Termination Competition (TermComp). Our tool demonstrates competitive performance against state-of-the-art termination provers, solving uniquely instances not solved by any existing tool.**

I. Introduction

Termination analysis of infinite-state systems is a fundamental problem in formal verification, with applications in static analysis, compiler optimisation, and the verification of safety-critical software. Proving termination and nontermination over LIA is generally undecidable [8], yet many practical techniques succeed on large classes of real-world programs. Modern techniques typically treat termination and nontermination as separate problems, since each requires a different witness.

State-of-the-art termination techniques centre around the synthesis of ranking functions for the transition relation [28], [22], [2], [19], [16], [30], [1]. The existence of a ranking function witnesses termination by providing a measure that strictly decreases along every execution. Alternative approaches are based on the computability of closed forms [15], [18], fixpoint computation [32], and the synthesis of disjunctively well-founded transition invariants [3], [12], [21], [31]. Nontermination analysis is usually executed separately from termination analysis. The most prevalent nontermination analysis technique is the detection of recurrent sets — sets of states from which execution cannot escape [1], [11], [14], [9], [10], [23], [4]. There also exist alternative approaches based on Hoare-style reasoning [24], safety proving [11], [4], [33], or representing infinite runs as sums of geometric series [25].

Most existing approaches treat termination and nontermination in isolation. In this work, we present a unified procedure that proves both termination and nontermination. This is done by extending and optimizing the safety-based nontermination analysis [11], allowing it to prove termination and nontermination of transition systems.

For termination analysis we implemented a new interpolation-based method for synthesising disjunctively well-founded transition invariants. This approach constructs invariants based on the termination conditions. The algorithm detects terminating traces, and constructs an interpolant that captures the reason why the trace necessarily terminates. Additionally, constructed transition invariant candidates are used to detect potentially nonterminating states that are reachable within the original system. These potentially nonterminating states guide the termination analysis, for more efficient construction of transition invariants and detection of recurrent sets.

Our new technique was implemented within the Golem CHC solver. We evaluated our technique over the Integer Transition Systems benchmarks from the Termination Competition. The combination of termination and nontermination approaches demonstrated significant improvement for nontermination analysis, while also solving a large number of terminating instances. The comparison with winners of Termination Competition 2025 (KoAT, LoAT [26], T2 [9]) also demonstrated that the developed approach is competitive with the state-of-the-art tools, and is able to solve unique instances, including instances that were never solved in the history of Termination competition.

Overall, this paper presents following contributions: (1) a new interpolation-based method for the synthesis of disjunctively well-founded transition invariants, (2) a unified termination/nontermination procedure, which utilises the intermediate results of analysis for self-guidance and (3) an implementation of the described procedure in Golem CHC solver and evaluation of proposed approach.

II. Preliminaries

This section introduces the definitions and formalisms used in the paper. Logical formulas in our approach are defined over the theory of Linear Integer Arithmetic (LIA). We use X, X' to denote set of variables (unprimed and primed), where $X' = \{v' | v \in X\}$, X is a set of state variables, and X' is a set of next state variables. If there are variables of many different states in the formula we use superscript notation, e.g., $X^{(0)}, \ldots, X^{(n)}$. s, s' are used to denote the specific states in the system.

The formula defined over X is a state formula, and a formula over $X \cup X'$ is a transition formula. State formulas encode a set of states, and transition formulas define binary relations over the state. Using $Id(X, X')$ we denote a transition formula $X = X'$

https://doi.org/

This article is licensed under a Creative Commons Attribution 4.0 International License

a) Transition systems and Safety Problem: A *transition system* (TS) is defined as $T = \langle Init, Tr, X \rangle$, where $Init$ is a formula encoding the initial states of the system, Tr is a transition formula, which encodes the transition relation, and X is a set of system's variables. With respect to transition system, a *transition trace* is a formula encoding multiple consecutive transitions: $Tr^n(X^{(0)}, \ldots, X^{(n)}) = Tr(X^{(0)}, X^{(1)}) \wedge \cdots \wedge Tr(X^{(n-1)}, X^{(n)})$. We write $Tr^n(X^{(0)}, \ldots, X^{(n)})$ for this conjunction throughout the paper. A *safety problem* is defined as $P = \langle Init, Tr, Bad, X \rangle$ where Bad is a formula that represents states that violate the safety property.

Safety verification amounts to determining whether there exists a transition trace that reaches a Bad state from the initial states. If TS is safe, then there exists an inductive invariant $Inv(X)$ s.t. (1) $\forall X.Init(X) \rightarrow Inv(X)$, (2) $\forall X.Inv(X) \wedge Tr(X, X') \rightarrow Inv(X')$, and (3) $\forall X.Inv(X) \rightarrow \neg Bad(X)$. We use $safetyCheck(P)$ to denote a safety verification procedure that returns a tuple $\langle SAFE, Inv(X) \rangle$ if TS is safe ($Inv(X)$ is inductive invariant). Otherwise, if TS is unsafe it returns $\langle UNSAFE, Tr^n(X^{(0)}, \ldots, X^{(n)}) \rangle$, such that:

$$\exists X^{(0)}, \ldots, X^{(n)}.Init(X^{(0)}) \wedge Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge Bad(X^{(n)})$$

b) Nontermination: A transition system is called **nonterminating** if there exists an initial state, from which it is possible to take any number of transitions:

$$\exists X.Init(X) \wedge \forall n \in \mathbb{N}. \exists X^{(1)}, \ldots, X^{(n)}.Tr^n(X, X^{(1)}, \ldots, X^{(n)}).$$

Definition 1 (Recurrent Set)**.** For a Transition System $\langle Init, Tr, X \rangle$, a formula $N(X)$ is a recurrent set if: (1) $\forall X.N(X) \rightarrow \exists X'.Tr(X, X') \wedge N(X')$, (2) $\exists X.N(X) \wedge Init(X)$.

Transition System is nonterminating if there exists a *recurrent set*. Intuitively, every state in the recurrent set has a successor that also lies in the recurrent set, and the recurrent set contains some of the initial states.

c) Termination: A TS is called **terminating** if for every initial state it is possible to take only limited number of transitions. For a Transition System $\langle Init, Tr, X \rangle$:

$$\forall X.Init(X) \rightarrow \exists n \in \mathbb{N}. \neg \exists X^{(1)}, \ldots, X^{(n)}.Tr^n(X, X^{(1)}, \ldots, X^{(n)}).$$

Definition 2 (Transition invariant)**.** For a Transition System $\langle Init, Tr, X \rangle$, $TrInv(X, X')$ is a transition invariant if: $\forall X, X'.R(X) \wedge Tr^*(X, X') \rightarrow TrInv(X, X')$, where $Tr^*(X, X')$ is the reflexive transitive closure of $Tr(X, X')$ and $R(X)$ is a formula encoding all the states, reachable within transition system.

Definition 3 (Well-founded Relation)**.** A binary relation $R \subseteq D \times D$ over a domain D is *well-founded* if there exists no infinite descending chain:

$$\nexists\, d_0, d_1, d_2, \cdots \in D.\ \forall i \in \mathbb{N}.\ (d_i, d_{i+1}) \in R$$

A relation $R(X, X')$ can be proven to be well-founded [28] by synthesizing a ranking function $f(X)$, such that $\forall X, X'.R(X, X') \rightarrow f(X) > f(X')$.

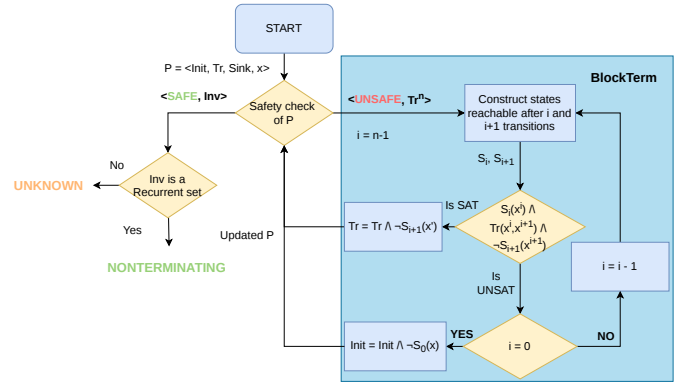


Figure 1. Execution flow of Safety-based Nontermination analysis, cf [11]

Transition invariant [29] can be used to prove termination. If transition invariant is a finite union of well-founded relations (for example if every separate relation has a ranking function), then the Transition System terminates.

Definition 4 (Disjunctively well-founded relation)**.** A relation $D(X, X') = \cup_{i=0}^{n} R_i(X, X')$ is called disjunctively well-founded if every relation $R_i(X, X')$ is well-founded.

d) Craig Interpolation: An abstraction technique [13], such that given two formulas A and B such that $A \wedge B$ is unsatisfiable, a Craig interpolant for A and B is a formula I such that $A \rightarrow I$, $I \wedge B$ is unsatisfiable, and I only contains the symbols, common to both A and B. We use $Itp(A, B)$ to denote the computation of the Craig interpolant for A and B.

A. Proving Nontermination via Safety

Many techniques reduce termination and nontermination analysis to safety verification. One notable approach is the reduction of nontermination to safety, which constructs strong recurrent sets via inductive invariants [11]. It was introduced in the context of proving nontermination of programs with loops, and can be extended for the Transition Systems. This approach is described below.

Definition 5 (Sink states)**.** For a Transition System $\langle Init, Tr, X \rangle$, a state s is a sink state, if $\neg \exists s'.Tr(s, s')$. We use $Sink(X)$ to denote the formula encoding all sink states in the transition system, such that $\forall X.Sink(X) \leftrightarrow \neg \exists X'.Tr(X, X')$

The Algorithm 1 for Safety-based Nontermination analysis (SNA) takes a safety problem $P = \langle Init, Tr, Sink, X \rangle$, where $Sink(X) = QE(\exists X'.\neg Tr(X, X'))$. As an output, it returns NONTERM if the system is nonterminating, and UNKNOWN, if the algorithm cannot conclude the analysis. The Figure 1 contains the execution flow of Safety-based Nontermination analysis.

The approach executes a safety check of P in a loop. At line 2, the algorithm handles the case when P is unsafe (i.e., exists a trace reaching sink states). In this case, the

Make a different wording.
Algorithm 1 intermediatE
We have this nice alg for nontermination
We can extract info from the run, and use it to find candidates for transition invariant.
3d Part - go beyond that - Termination and Nontermination dont communicate -
we can use it.

Take a nonterminating TS for example 1. Now add a small modification of TS. Now it is terminating
However it generates transition traces(Tr^n), that can be used to generate TrInvs

Algorithm 1: Nontermination analysis using Safety (SNA), cf [11]

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$
Output: NONTERM | UNKNOWN

1 **while** $\top$ **do**
2 $\quad \langle res, \phi = Tr^n \text{ or } Inv\rangle \leftarrow safetyCheck(P)$
3 $\quad$ **if** $res = UNSAFE$ **then**
4 $\quad\quad P \leftarrow$ BLOCKTERM(P, ϕ)
5 $\quad$ **else**
6 $\quad\quad$ **if** $\forall X.\phi(X) \rightarrow \exists X'.Tr(X, X')$ **then**
7 $\quad\quad\quad$ **return** NONTERM
8 $\quad\quad$ **return** UNKNOWN

algorithm calls the BLOCKTERM procedure (Algorithm 2), which refines the safety problem P by blocking the states that determenistically lead to the sink states, updating the Safety problem. BLOCKTERM is described in detail below. In the case when P is safe, algorithm checks if inductive invariant $Inv(X)$ is a recurrent set (line 5). The check at line 5 can be done, using quantifier elimination to eliminate X', checking if the following formula is unsatisfiable:

$$\exists X.\phi(X) \wedge \neg QE(\exists X'.Tr(X, X')).$$

If $Inv(X)$ is a recurrent set, then the transition system is nonterminating. Otherwise, algorithm returns UNKNOWN, as the analysis is inconclusive.

Algorithm 2: BLOCKTERM (P, Tr^n), cf [11]

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$, terminating trace $Tr^n(X^{(0)}, \ldots, X^{(n)}) = Tr(X^{(0)}, X^{(1)}) \wedge \cdots \wedge Tr(X^{(n-1)}, X^{(n)})$
Output: Refined safety problem $P' = \langle Init', Tr', Sink', X\rangle$
Data : $S_0, \ldots, S_n$ - formulas which encode states within trace reachable in 0, ..., n transitions

1 $i \leftarrow n - 1$
2 **while** $i > 0$ **do**
3 $\quad S_{i+1}(X^{(i+1)}) \leftarrow QE(\exists X^{(0)}, \ldots, X^{(i)}, X^{(i+2)}, \ldots, X^{(n)}. Init(X^{(0)}) \wedge Tr^n \wedge Sink(X^{(n)}))$
4 $\quad S_i(X^{(i)}) \leftarrow QE(\exists X^{(0)}, \ldots, X^{(i-1)}, X^{(i+1)}, \ldots, X^{(n)}. Init(X^{(0)}) \wedge Tr^n \wedge Sink(X^{(n)}))$
5 $\quad$ **if** *SAT ?* $S_i(X) \wedge Tr(X, X') \wedge \neg S_{i+1}(X')$ **then**
6 $\quad\quad Tr'(X, X') \leftarrow Tr(X, X') \wedge \neg S_{i+1}(X')$
7 $\quad\quad$ **return** $\langle Init, Tr', Sink, X\rangle$
8 $\quad i \leftarrow i - 1$
9 $Init'(X^{(0)}) \leftarrow Init(X^{(0)}) \wedge \neg QE(\exists X^{(1)}, \ldots, X^{(n)}.Init(X^{(0)}) \wedge Tr^n \wedge Sink(X^{(n)}))$
10 **return** $\langle Init', Tr, Sink, X\rangle$

The BLOCKTERM procedure takes as input a safety problem P and a trace formula $Tr^n(X^{(0)}, \ldots, X^{(n)})$ such that $Init(X) \wedge Tr^n(X, \ldots, X') \wedge Sink(X')$ is satisfiable. The algorithm iteratively analyses the transition trace from the sink. It checks if it is possible to take a transition that does not lead to the sink states. Line 4 checks if $S_i(X) \wedge Tr(X, X') \wedge \neg S_{i+1}(X')$ is satisfiable, sending a query to the SMT solver. If it is possible, then the algorithm blocks the states that deterministically lead to the sink states at line 6. Otherwise, if such states do not exist, the algorithm blocks the subset of initial states that deterministically lead to the sink states (line 9).

Lemma 1. *If Algorithm 1 returns NONTERM, then the transition system is nonterminating, cf [28].*

Example 1. Consider a transition system $TS = \langle Init, Tr, X\rangle$ with boolean variables $X = \{x\}$ and: $Init(X) \triangleq \top$, $Tr(X, X') \triangleq (x' = x - 1 \vee x' = x - 2) \wedge x > 0$. The sink states are computed as $Sink(X) \triangleq x \leq 0$. The initial safety problem is $P = \langle Init, Tr, Sink, X\rangle$. This transition system is trivially terminating, however, Algorithm 1 would not be able to detect the termination of this example. It would loop infinitely, blocking initial states that lead to termination.

Note. The termination of this system can be proven with a simple well-founded transition invariant $TrInv(X, X') = x' < x \wedge x > 0$

III. TERMINATION ANALYSIS VIA INTERPOLATION-BASED TRANSITION INVARIANT GENERATION

We propose a new approach for proving termination that leverages safety verification to generate a well-founded transition invariant candidate via interpolation. At each iteration, our procedure constructs a trace from the initial states to a sink state. A well-founded transition invariant is then constructed based on the overapproximation of the terminating trace. If the invariant covers all reachable states, termination is concluded. Additionally, the algorithm uses the candidate transition invariant to refine the state space, focusing on states where transition invariant is not valid. The interpolation-based approach enables efficient property-directed transition invariant generation, as it is constructed with respect to the termination condition.

A. Interpolation-based Generation of Well-Founded Transition Invariants

The core of our approach is the procedure for automated, interpolation-based generation of well-founded transition invariants: TRINVSYNTHESIS. Given a transition system and a terminating trace, the procedure constructs a disjunctively well-founded approximation of the transition relation. If the resulting formula constitutes a transition invariant over all reachable states, it witnesses termination.

The Procedure consists of five major steps: (1) computing NT^n, the formula encoding states that do not necessarily terminate within n transitions. NT^n is used to construct the interpolant. The negation of NT^n defines the states that necessarily reach $Sink$ in at most n transitions; (2) constructing an over-approximation of the terminating trace via interpolation. The usage of sink states allows the interpolant to capture the reason for termination; (3) converting the interpolant into an underapproximating DNF formula. This allows to simplify synthesis of the ranking function for the disjuncts, as every disjunct is a conjunction; (4) filtering disjuncts to retain only those encoding well-founded relations. Transition invariant

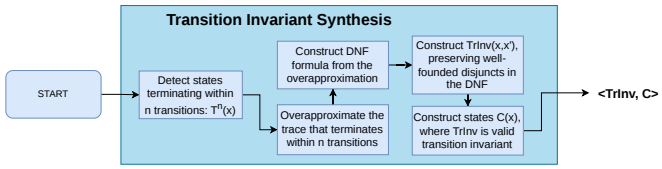


Figure 2. Execution Flow of Interpolation-based Transition Invariant Generation

Algorithm 3: TRINVSYNTHESIS$(P, Tr^n, TInv)$.

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$, transition trace $Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)})$, candidate transition invariant $TInv(X, X')$

Output: Updated transition invariant $TInv(X, X')$ and formula $C(X)$ encoding states covered by transition invariant

1 $NT^n(X^{(0)}) \leftarrow QE(\exists X^{(1)}, \ldots, X^{(n)}.$
$Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)}))$

2 $itp(X, X') \leftarrow$
$Itp(Tr^n(X, \ldots, X'), \neg NT^n(X) \wedge \neg Sink(X'))$

3 $Cs(X, X') \leftarrow toDNF(itp)$

4 $TInv \leftarrow TInv \vee \bigvee_i \{d_i \in Cs | \exists$ ranking function $f(X)\}$

5 $C(X) \leftarrow \neg QE(\exists X', X''.\ \big(Id(X, X') \vee TInv(X, X')\big) \wedge$
$Tr(X', X'') \wedge \neg TInv(X, X''))$

6 **return** $\langle TInv, C\rangle$

needs to be disjunctively well-founded to witness termination; and (5) computing the formula encoding states for which the produced formula is a valid transition invariant. Transition invariant needs to be valid only over the reachable states to conclude termination, which weakens the properties that need to hold for transition invariant. The overall flow of the procedure is illustrated in the Figure 2.

Algorithm 3 takes as an input a safety problem $P = \langle Init, Tr, Sink, X\rangle$, a transition trace $Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)})$, and a transition invariant formula $TInv(X, X')$. It returns a tuple of formulas $\langle TInv, C\rangle$, where $TInv(X, X')$ is a disjunctively well-founded transition invariant candidate, and $C(X)$ is a formula encoding the states for which $TInv$ is a valid transition invariant. It proceeds in the following steps:

Step 1: *Computing NT^n (line 1).* $NT^n(X)$ encodes all states guaranteed to reach a sink state within n transitions, is defined by the following property:

$$\exists X^{(0)}, \ldots, X^{(n)}. NT^n(X^{(0)}) \wedge Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)})$$

NT^n is constructed with quantifier elimination, eliminating all variables from the formula except for $X^{(0)}$. Construction of $NT^{(n)}$ enables the overapproximation of the trace.

Example 2. Consider $TS = \langle Init, Tr, X\rangle$ with $Init \triangleq v > 0$, $Tr(X, X') \triangleq ((v' = v - 1 \vee v' = v - 2) \wedge v > 0) \vee (v < -10 \wedge v' = v - 1)$, $X \triangleq \{v\}$. For $n = 2$ quantifier elimination would yield $NT^2(X) = v > 2$. States with $v \in \{3, 4\}$ are not necessarily reaching $Sink$ in two transitions but can do so, therefore, they are included in NT^2.

Step 2: *Interpolation (line 2).* The following formula is unsatisfiable by construction, since any state in $\neg NT^n$ must reach $Sink$ within n steps:

$$\neg NT^n(X^{(0)}) \wedge Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)})$$

This allows to compute the **Craig interpolant**, which over-approximates the trace Tr^n. Crucially, the usage of the sink states for interpolant construction allows the interpolant to capture the reason *why* the particular trace is leading to the termination.

Step 3: *DNF underapproximation (line 3).* For transition invariant to be a proof of termination, it must be disjunctively well-founded, meaning every disjunct must encode a well-founded relation. Full conversion to a DNF format is expensive and can cause an exponential blowup in formula size. Additionally, even full DNF conversion does not guarantee that the resulting formula would witness termination, or even be a transition invariant. Our algorithm follows a lazy approach to DNF conversion, constructing an underapproximating DNF formula on demand, limiting the number of disjuncts in the resulting formula by a tunable constant. The disjuncts are constructed via syntactic processing of the formula, unwrapping the innermost disjunctions and conjunctions. [**Konstantin**: *To be slightly rewritten.*] It allows to significantly reduce the size of the resulting formula and speed up the DNF conversion, while preserving correctness: in the worst case, the resulting formula fails to be a transition invariant. Empirically, we observed that limiting the number of disjuncts results in more size-efficient and quick termination proofs.

Step 4: *Filtering Well-Founded Disjuncts (line 4).* For each disjunct $d(X, X') \in Cands(X, X')$ algorithm checks if $d(X, X')$ encodes a well-founded relation. Our technique uses automated linear ranking function synthesis [28] for checking the existence of a ranking function. The disjuncts for which a ranking function exists are preserved, and the rest are discarded. The resulting formula is disjoined with the input candidate transition invariant $TInv$.

Step 5: *Synthesis of the states for which transition invariant is valid (line 5).* If $TrInv$ is a transition invariant over all reachable states, then TS terminates, since every disjunct corresponds to well-founded relation. To verify this, procedure computes the formula $C(X)$ encoding the states over which $TrInv$ is a valid transition invariant.

Definition 6 (Coverage formula)**.** Consider a transition system $TS = \langle Init, Tr, X\rangle$ and a formula $TrInv(X, X')$. We say that $C(X)$ is a coverage formula, if the following properties hold:

$$\forall X, X'. C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$
$$\forall X, X', X''. C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'')$$

Meaning, that $TrInv$ is a transition invariant over states satisfying $C(X)$.

Similarly to NT^n, $C(X)$ is constructed using the quantifier elimination, negating the states for which $TrInv$ is not a transition invariant.

Example 3. Continuing Example 2, suppose the procedure constructs $TrInv \triangleq v' < v \wedge v' \geq 0$. Quantifier elimination then yields $C(X) \equiv v \geq 0$.

Note. If $\neg C(X)$ is not reachable, then $TrInv(X, X')$ is a disjunctively well-founded transition invariant over all reachable states, and witnesses the termination of transition system. For illustration, transition invariant in the Example 3 is sufficient to prove termination, since $\neg C(X) \triangleq v < 0$ is not reachable in the transition system.

Lemma 2. *Let $TS = \langle Init, Tr, X \rangle$ be a transition system and let $C(X)$ be a formula such that $\neg C(X)$ is unreachable in TS. If $TrInv(X, X')$ is a finite disjunction of formulas encoding well-founded relations and satisfies:*

$$\forall X, X', X''.C(X) \wedge (TrInv(X, X') \vee Id(X, X')) \\ \wedge Tr(X', X'') \rightarrow TrInv(X, X'')$$

then TS is terminating.

Proof. We show that $TrInv$ is a disjunctively well-founded transition invariant over the reachable states of T, from which termination follows by [29].

Well-foundedness. By construction, for each disjunct d_i of $TrInv = \bigvee_i d_i$ there exists a ranking function. Therefore, every disjunct is a logical encoding of well-founded relation [28]. Hence $TrInv$ is disjunctively well-founded.

Transition invariant. We show that $\forall X, X'.\ R(X) \wedge Tr^*(X, X') \rightarrow TrInv(X, X')$, where $R(X)$ denotes the reachable states of T, by induction on the number of steps n in $Tr^*(X, X')$.

Base case ($n = 1$): It is needed to prove that:

$$R(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$

Since $\neg C(X)$ is unreachable, $R(X) \rightarrow C(X)$.

The following assumption holds originally by the lemma statement:

$$C(X) \wedge Id(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'')$$

Since $Id(X, X')$ is an identity relation, it can be simplified to:

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X').$$

Since $R(X) \rightarrow C(X)$, therefore $R(X) \wedge Tr(X, X') \rightarrow C(X) \wedge Tr(X, X')$, the base case holds.

Inductive step: Assume $R(X) \wedge Tr^n(X, \ldots, X') \rightarrow TrInv(X, X')$ for some $n \geq 1$. Let $Tr^{n+1}(X, X'')$ hold, so there $\exists X, \ldots, X', X''.R(X) \wedge Tr^n(X, \ldots, X') \wedge Tr(X', X'')$. Since $R(X) \wedge Tr^n(X, \ldots, X') \rightarrow TrInv(X, X')$, then trivially $R(X) \wedge Tr^n(X, X') \wedge Tr(X', X'') \rightarrow R(X) \wedge TrInv(X, X') \wedge Tr(X', X'')$. Now, due to the fact that $R(X) \rightarrow C(X)$, the following holds: $R(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow C(X) \wedge TrInv(X, X') \wedge Tr(X', X'')$. And by claim in the lemma:

$$C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'').$$

Therefore, through the chain of implications, we have:

$$R(X) \wedge Tr^{n+1}(X, X') \rightarrow TrInv(X, X').$$

Thus $TrInv$ is a disjunctively well-founded transition invariant over all reachable states, and TS is terminating. □

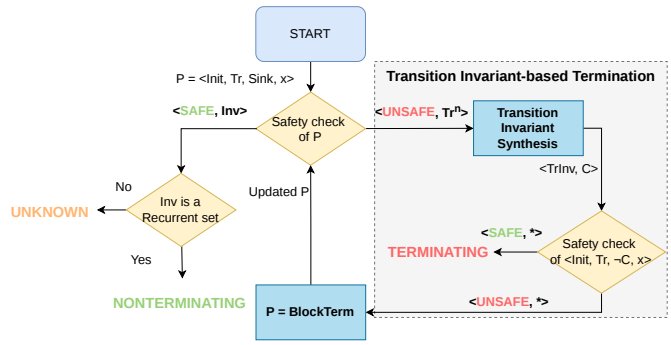


Figure 3. Execution Flow of Interpolation-based Transition Invariant Generation (New components are placed in a grey block, lines 5,6,10).

Algorithm 4: Interpolation-based Termination Analysis.

Termination/Nontermination (based on the title)
Use undersccre

Input : Safety problem $P = \langle Init, Tr, Sink, X \rangle$
Output: TERM | NONTERM | UNKNOWN
Data : $TInv$ - formula that encodes transition invariant

1 $TInv \leftarrow \bot$
2 **while** $\top$ **do**
3 $\quad \langle res, \phi = Tr^n \text{ or } Inv \rangle \leftarrow safetyCheck(P)$
4 $\quad$ **if** $res = UNSAFE$ **then**
5 $\quad\quad \langle TInv, C \rangle \leftarrow$ TRINVSYNTHESIS$(P, Tr^n, TInv)$
6 $\quad\quad \langle res', * \rangle \leftarrow safetyCheck(\langle Init, Tr, \neg C, X \rangle)$
7 $\quad\quad$ **if** $res' = UNSAFE$ **then**
8 $\quad\quad\quad P \leftarrow$ BLOCKTERM(P, ϕ)
9 $\quad\quad$ **else**
10 $\quad\quad\quad$ **return** $TERM$
11 $\quad$ **else**
12 $\quad\quad$ **if** $\forall X.\phi(X) \rightarrow \exists X'.Tr(X, X') \wedge \phi(X')$ **then** **return** $NONTERM$
13 $\quad\quad$ **return** UNKNOWN

B. Interpolation-based Termination Analysis

This subsection presents the full termination analysis procedure, using the interpolation-based transition invariant generation. It integrates TRINVSYNTHESIS into the safety-based analysis loop of Algorithm 1. At each iteration, instead of only blocking terminating states, the algorithm also constructs a transition invariant candidate. The overall flow is shown in Figure 3.

The Algorithm 4 takes as input a Safety Problem $P = \langle Init, Tr, Sink, X \rangle$, where $Init$ and Tr encode the initial states and transition relation, $Sink$ is the formula encoding sink states, and X is a set of variables over which the

Transition System is defined. As an output, the algorithm returns NONTERM if TS is nonterminating, TERM if it is terminating, and UNKNOWN if the procedure cannot determin the final result. The procedure maintains a formula encoding a candidate transition invariant $TInv(X, X')$. During the execution, $TInv$ is extended with new disjuncts, which are generated by the TRINVSYNTHESIS procedure, and encode well-founded relations.

Algorithm commences by analyzing the original safety problem. Every iteration, similarly to the Algorithm 1, begins with the safety check at line 2. The algorithm handles two possible cases:

a) UNSAFE.: A terminating trace Tr^n exists. The algorithm calls TRINVSYNTHESIS to extend $TInv$ and compute the coverage formula $C(X)$ (line 5). Then, it checks if $\neg C(X)$ is reachable from the initial states (line 6). If $\neg C(X)$ is reachable, algorithm refines the safety problem by blocking the states deterministically leading to termination via BLOCKTERM, and the loop continues. If it is not reachable, then $TInv$ is a transition invariant over all reachable states, and algorithm concludes termination.

b) SAFE.: No sink state is reachable. If $\phi(X)$ is a recurrent set, the system is nonterminating. Otherwise, the algorithm returns UNKNOWN as the analysis is inconclusive.

Example 4. Consider TS from Example 2 with $Init \triangleq v > 0$, $Tr \triangleq \big((v' = v - 1 \vee v' = v - 2) \wedge v > 0\big) \vee \big(v < -10 \wedge v' = v - 1\big)$, $X \triangleq \{v\}$.

Preprocessing computes $Sink \equiv v \leq 0 \wedge v \geq -10$. Then, the algorithm is called with $P = \langle Init, Tr, Sink, X\rangle$.

The first safety check detects that state satisfying $Sink$ exist within $Init(X)$. Constructed trace is empty, therefore constructed transition invariant and covered states are $TInv \triangleq \bot$ and $C(X) \triangleq \bot$. Algorithm proceeds to BLOCKTERM, updating init states:

$$Init(X) \leftarrow v > 0 \vee v < -10$$

On the next iteration, a trace Tr^1 reaching $Sink$ is found. TRINVSYNTHESIS produces a transition invariant candidate $TInv(X, X') \triangleq v' \leq v - 1 \wedge v' \geq 0$ and the formula $C(X) = v \geq 0$. The safety check for $\langle Init, Tr, \neg C(X), v\rangle$ returns SAFE, and algorithm concludes that the transition system is terminating.

C. Correctness

This subsection proves the soundness of the introduced approach. We first prove that when algorithm returns TERM, the transition system is nonterminating, and then we prove the correctness of the nontermination.

Lemma 3. *If Algorithm 4 returns $TERM$, then the transition system $\langle Init, Tr, X\rangle$ is terminating.*

Proof. $TERM$ is returned only in the line 10, of the algorithm. It means that there exists a transition invariant $TInv$ which is valid over states $C(X)$, and $\neg C(X)$ is unreachable

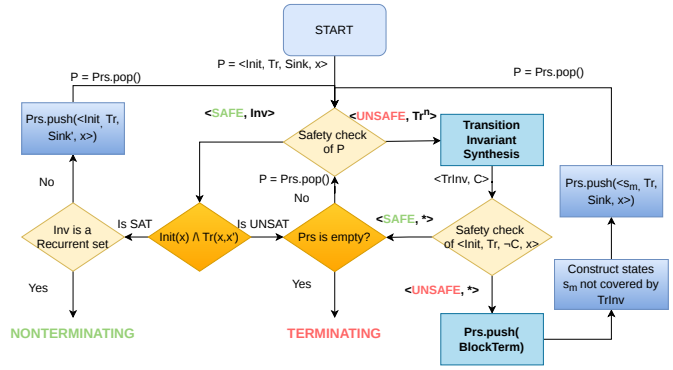


Figure 4. Optimized Interpolation-based Termination Analysis (New components are highlighted in dark orange and dark blue colours).

from the initial states. The safety problem is updated iteratively, blocking the states that deterministically lead to the termination, so in the end the $P = \langle Init', Tr', Sink, X\rangle$, where $Init'(X) = Init(X) \wedge \bigwedge_{i=0}^{m} \neg S_i(X)$ and $Tr' = Tr(X) \wedge \bigwedge_{i=0}^{l} \neg S_j(X')$. According to the Lemma 2, Transition System $\langle Init', Tr', X\rangle$ is terminating, since $TInv$ is a disjunctively well-founded transition invariant over all reachable states.

Since the states satisfying $\bigvee_{i=0}^{m} S_i(X)$ and $\bigvee_{i=0}^{l} S_j(X')$ deterministically lead to the termination, every trace of $\langle Init, Tr, X\rangle$ either follows Tr' or passes through a blocked state which leads to termination. Hence $\langle Init, Tr, X\rangle$ is terminating. □

Lemma 4. *If Algorithm 4 returns $NONTERM$, then the transition system $\langle Init, Tr, X\rangle$ is nonterminating.*

Proof. The proof of this lemma follows directly from the lemma 1, as the nontermination proof procedure directly corresponds to one described in the Algorithm 1. □

IV. INTERPOLATION-BASED TERMINATION ANALYSIS: OPTIMIZATIONS

Algorithm 4 is already capable to prove termination efficiently, via the construction of transition invariants. This section describes three optimizations that extend its power: a termination check, safety problem refinement, and targeted analysis of states not covered by the transition invariant. The overall flow of the optimized algorithm is shown in Figure 4.

A. Termination Check

When the safety check returns SAFE and $Inv(X)$ is not a recurrent set, the algorithm can additionally check whether it is possible to take transition from the initial states by querying:

$$Init(X) \wedge Tr(X, X')$$

If this formula is unsatisfiable, no transition can be taken from any initial state (or all transitions are blocked, meaning all reachable states deterministically lead to termination) and the system is trivially terminating.

Algorithm 5: Optimized Interpolation-based Termination Analysis.

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$
Output: TERM | NONTERM
Data : $TInv$ - formula that encodes transition invariant, Prs - stack of Safety Problems to be analyzed for termination

1 $TInv \leftarrow \bot$
2 $Prs \leftarrow [P]$
3 **while** $\top$ **do**
4 $\quad \langle Init', Tr', Sink', X\rangle \leftarrow Prs.pop()$
5 $\quad \langle res, \phi = Inv \text{ or } Tr^n\rangle \leftarrow safetyCheck(\langle Init', Tr', Sink', X\rangle)$
6 $\quad$ **if** $\langle res, \phi\rangle = \langle UNSAFE, Tr^n(X^{(0)}, \ldots, X^{(n)})\rangle$ **then**
7 $\quad\quad \langle TInv, C\rangle \leftarrow$ TrInvSynthesis$(P, \phi, TInv)$
8 $\quad\quad \langle res', *\rangle \leftarrow safetyCheck(\langle Init', Tr', \neg C, X\rangle)$
9 $\quad\quad$ **if** $res' = SAFE$ **then**
10 $\quad\quad\quad$ **if** $Prs = \emptyset$ **then return** $TERM$
11 $\quad\quad$ **else**
12 $\quad\quad\quad Prs.push($BlockTerm$(P', \phi))$
13 $\quad\quad\quad S_m \leftarrow QE(\exists X^{(0)}, \ldots, X^{(m-1)}, Init(X^{(0)}) \wedge \phi(X^{(0)}, \ldots, X^{(m)}) \wedge \neg C(X^{(m)}))$
14 $\quad\quad\quad Prs.push(\langle S_m, Tr', Sink', X\rangle)$
15 $\quad$ **else**
16 $\quad\quad$ **if** *UNSAT ?* $\phi(X) \wedge Tr(X, X')$ **then return** $TERM$
17 $\quad\quad$ **if** $\forall X.\phi(X) \rightarrow \exists X'.Tr(X, X') \wedge \phi(X')$ **then return** $NONTERM$
18 $\quad\quad Sink'(X) \leftarrow \neg QE(\exists X'.Tr(X, X'))$
19 $\quad\quad Prs.push(\langle Init', Tr', Sink', X\rangle)$

B. Safety Problem Refinement

When $Inv(X)$ is not a recurrent set, this is often because BlockTerm has eliminated all transitions leading to $Sink$, leaving states in Inv with no outgoing transition. These states are themselves sink states under the refined Tr' and can be computed as:

$$Sink'(X) = \neg QE(\exists X'.Tr'(X, X'))$$

The formula $Sink'$ replaces $Sink$ in the safety problem, enabling the analysis to continue rather than returning UNKNOWN.

C. Refinement of transition invariant

Constructed transition invariant can be used to guide the termination analysis. Particularly, the algorithm constructs the formula $C(X)$ encoding states for which the transition invariant is valid. If $\neg C(X)$ is reachable, it means that there exist states for which the transition invariant is not valid, and which potentially can lead to non-termination. The intuition is to compute the reachable states S_m within $\neg C(X)$ to construct transition invariant covering those states in particular, or to detect a recurrent set otherwise.

D. Full Algorithm

Algorithm 5 extends Algorithm 4 with three changes corresponding to the optimizations above. First, in addition to the transition invariant, the algorithm maintains a list of safety problems which need to be analyzed: Prs (line 2). Each iteration, the algorithm pops a safety problem from the list and analyzes it. This allows the algorithm to concentrate on the termination analysis of particular reachable states within TS, to efficiently expand the transition invariant. The original safety problem is always present in the Prs (with blocked states that deterministically lead to termination).

Second, when $\neg C$ is reachable, the algorithm pushes both the BlockTerm-refined problem and a new problem focused on the states, not covered by transition invariant S_m (line 14). Thus allowing to concentrate on the termination analysis of particular reachable states, instead of necessarily proving termination for the whole system.

Third, additional changes were made in the SAFE case, where algorithm checks for the termination at line 16, and refines the safety problem with the new sink states instead of returning UNKNOWN (line 18).

Lemma 5. *If Algorithm 5 returns $TERM$, then the original transition system $\langle Init, Tr, X\rangle$ is also terminating.*

Proof. TERM is returned at line 10 or line 16. By construction, to return $TERM$, the termination conditions need to be satisfied for the transition system $\langle Init', Tr', X\rangle$, where $Init'(X) = Init(X) \wedge \bigwedge_j \neg S_i(X)$, and $Tr'(X, X') = Tr(X, X') \wedge \bigwedge_j \neg S_j(X')$. $\bigvee_{i=0}^{l} S_i(X)$ and $\bigvee_{j=0}^{k} S_j(X')$ are the states that deterministically lead to termination.

Case 1. If $TERM$ is returned at line 10, then the termination proof follows directly from the Lemma 2.

Case 2. If $TERM$ is returned at line 16, then the formula $Init'(X) \wedge Tr'(X, X')$ is unsatisfiable, so no transition is reachable from $Init'$. It means that the transition system is terminating, since all feasible transitions in the original TS lead to deterministically terminating states. □

Lemma 6. *If Algorithm 5 returns $NONTERM$, then the original transition system $\langle Init, Tr, X\rangle$ is nonterminating.*

Proof. NONTERM is returned at line 17, where $Inv(X)$ is a recurrent set for the current refined system $\langle Init', Tr', X\rangle$. By construction, $Init'$ is reachable from $Init$ (it is either $Init$ itself or derived from a trace starting in $Init$), and $Tr'(X, X') \implies Tr(X, X')$. Based on the Lemma 1, if $NONTERM$ is returned, then there exists a recurrent set in the transition system $\langle Init', Tr', X\rangle$, which means that there also exists a recurrent set in the original transition system $\langle Init, Tr, X\rangle$. It proves nontermination. □

V. Implementation and Evaluation

This section presents implementation details and describes the experimental evaluation of our termination approach.

A. Implementation

We implemented Algorithms 1, 4, and 5 inside of the Golem [5]; we refer to these implementations as SNA, SNA+Term and InvGuidance. SNA handles trivial termination cases during preprocessing, such as the absence of initial states or the absence of transitions from initial

states. GOLEM supports multiple different safety verification procedures, such as Property Directed Reachability [7] (PDR), Transition Power Abstraction [6] (TPA) and Interpolation-based Model Checking [27]. Additionally, we use quantifier elimination, already implemented in GOLEM.

For SMT queries and interpolant construction, our implementation uses the OPENSMT SMT solver [20]. OPENSMT is integrated within GOLEM and provides native support for interpolation.

B. Evaluation

We compare INVGUIDANCE against SNA and SNA+TERM, to evaluate the effect of combination of the termination and nontermination techniques. Additionally, we evaluate INVGUIDANCE against several state-of-the-art tools: LOAT [26] (version eacb74b), KOAT [26] (version 41cb681), T2 [9] (version 10f1373). Evaluation focuses on termination analysis of Integer Transition Systems. Experiments were conducted on Ubuntu 20.04 machine with an AMD EPYC 7452 32-core processor and 8x32 GiB RAM. Our evaluation addresses the following research questions:

- **RQ1:** How does extending SNA with the termination procedure affect verification performance?
- **RQ2:** How does INVGUIDANCE perform compared to state-of-the-art tools?

The benchmark suite of Integer Transition Systems is taken from the Termination Competition. Overall there are 1222 benchmarks (538 nonterminating, 665 terminating, and 19 unknown). The evaluation was run with 120 seconds timeout (increase of the timeout almost does not influence the evaluation).

Exclude 42 benchmarks from the evaluation

Interpolation-based Transition Invariant Generation for Proving and Disproving Termination

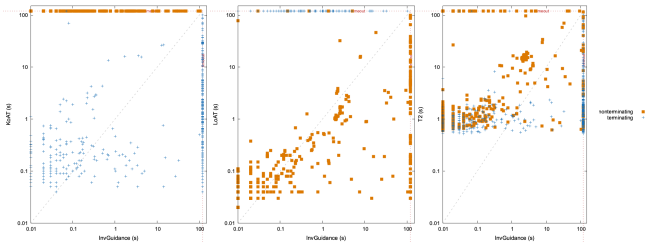
ItpTIG ItpTig+ (in the supersctipy)

Algorithm	Terminating	Nonterminating	Solved Uniquely
SNA	42	343	5
SNA+TERM	228	335	17
INVGUIDANCE	343	442	283

Table I
COMPARISON OF INVGUIDANCE WITH SNA AND SNA+TERM.

Table **??** gives the experimental results for INVGUIDANCE, SNA, and SNA+TERM. Our results indicate that INVGUIDANCE consistently outperforms both SNA and SNA+TERM for Terminating and Nonterminating instances. We attribute this to the usefulness of creating "subproblems" in the INVGUIDANCE, that allows the algorithm to focus on the termination of specific subproblem instead of considering the whole system. This benefits recurrent set detection, by focusing the analysis on specific states not covered by the transition invariant. It also benefits termination proving, by enriching the transition invariant with additional disjuncts that cover different regions of the reachable state space. In answer to **RQ1**, the combination of termination and nontermination reasoning in INVGUIDANCE yields significantly improved performance across both problem classes.

Figure 5 shows a scatter plot comparing INVGUIDANCE against state-of-the-art termination analysers. Orange squares



Increase the font size

Figure 5. Comparison of INVGUIDANCE with state-of-the-art tools.

denote nonterminating instances; blue crosses denote terminating instances. Points above the diagonal indicate instances where INVGUIDANCE is faster than the competitor. INVGUIDANCE is able to solve 785 benchmarks, with T2 solving 1044, KOAT 604, and LOAT 527 instances. As shown in Figure 5, INVGUIDANCE is competitive with state-of-the-art termination analysers. Among all tools, INVGUIDANCE uniquely solves 8 instances. Notably, 2 of these — `juLinkedListCreateAddAll.jar-obl-11` and `juLinkedListCreateAddAllAt.jar-obl-17` — belong to the 19 previously unsolved instances in the Termination Competition, for which INVGUIDANCE finds recurrent sets proving nontermination. In answer to **RQ2**, INVGUIDANCE is competitive with state-of-the-art tools overall and uniquely solves instances previously unsolved in the Termination Competition, demonstrating the practical value of the interpolation-based transition invariant approach.

Total

Algorithm	Terminating	Nonterminating	Solved Uniquely
LOAT	2	525	48
KOAT	604	0	26
T2	614	430	40
INVGUIDANCE	343	442	8

Table II
COMPARISON OF INVGUIDANCE WITH STATE-OF-THE-ART TOOLS

VI. RELATED WORK

Termination and nontermination analysis are very active areas of research. We will discuss techniques, related to Interpolation-based Termination Analysis below.

Termination. There exist many different techniques for the termination analysis of software. One of the most researched and applied techniques iss the synthesis of the ranking functions for the transition relation, which allows to prove termination [28], [22], [2], [19], [16], [30], [1]. State of the art techniques are centered on the construction of multiphase or lexicographic ranking functions. These complex techniques significantly broaden the class of programs that ranking functions can handle. However, since they reason on the original transitions, the complexity of original system can make construction of ranking function significantly more complex, or even impossible.

Another techniques, which allows to efficiently prove termination are fixpoint computation [32] and the computation of closed forms [15], [18].

There also exists another set of techniques which can prove termination that are based on the construction of well-founded transition invariants [12], [21], [31]. These approaches allow to approximate the behaviour of the transition system, and if the approximation is calculated succesfully the termination analysis is significantly simplified. The core problem of these approaches is the synthesis of transition invariant. Majority of transition invariant-based techniques are based on the synthesis of transition invariants via template synthesis, which is not guided by the termination conditions and can often diverge and sometimes even connot produce a descriptive invariant.

Nontermination. Nontermination analysis is typically centered around the construction of recurrent sets — sets of states from which execution cannot escape [1], [14], [9], [11], [10], [23]. These techniques can vary from the template-based synthesis to the safety-check based recurrent set construction. Another set of techniques are based on the safety-based termination analysis [4], [33], that allow to use safety provers for the proof of nontermination. An alternative family of techniques detects nontermination by identifying lasso-shaped executions — finite paths leading to a loop that can be repeated indefinitely [4], [17], [25].

VII. Conclusion

This paper introduces Termination Analysis via Interpolation-based Synthesis of Transition Invariants. A novel approach for the termination analysis that extends Safety-based Nontermination Analysis technique. The key insight of our approach is to use interpolation for the construction of termination witnesses, that allows the transition invariant to be guided by termination condition. Additionally, our technique refines the state space, reusing the intermediate results of termination and nontermination analysis, concentrating on states that can be potentially nonterminating. We also optimized and adapted the original Nontermination analysis technique. Usage of safety verifiers provides a strong background for the analysis, allowing to quickly check termination and nontermination. We proved the soundness of the algorihtm and conducted an extensive experimental evaluation, which demonstrates both the eficiency of the combination of termination and nontermination techniques, and the competitive performance compared to other state-of-the-art termination analysis tools.

References

[1] Ben-Amram, A.M., Doménech, J.J., Genaim, S.: Multiphase-linear ranking functions and their relation to recurrent sets. In: Chang, B.E. (ed.) Static Analysis - 26th International Symposium, SAS 2019, Porto, Portugal, October 8-11, 2019, Proceedings. pp. 459–480. Lecture Notes in Computer Science, Springer (2019). https://doi.org/10.1007/978-3-030-32304-2_22, https://doi.org/10.1007/978-3-030-32304-2_22

[2] Ben-Amram, A.M., Genaim, S.: Ranking functions for linear-constraint loops. J. ACM **61**(4), 26:1–26:55 (2014). https://doi.org/10.1145/2629488, https://doi.org/10.1145/2629488

[3] Beyer, D., Jankola, M., Lingsch-Rosenfeld, M.: Transition invariants revisited: Termination witnesses and their validation. In: Proc. CAV. Springer (2026), https://www.sosy-lab.org/research/pub/2026-CAV.Transition_Invariants_Revisited_Termination_Witnesses_and_Their_Validation.pdf

[4] Biere, A., Artho, C., Schuppan, V.: Liveness checking as safety checking. In: Cleaveland, R., Garavel, H. (eds.) 7th International ERCIM Workshop in Formal Methods for Industrial Critical Systems, FMICS 2002, ICALP 2002 Satellite Workshop, Málaga, Spain, July 12-13, 2002. pp. 160–177. No. 2 in Electronic Notes in Theoretical Computer Science, Elsevier (2002). https://doi.org/10.1016/S1571-0661(04)80410-9, https://doi.org/10.1016/S1571-0661(04)80410-9

[5] Blicha, M., Britikov, K., Sharygina, N.: The golem horn solver. In: Enea, C., Lal, A. (eds.) Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 13965, pp. 209–223. Springer (2023), https://doi.org/10.1007/978-3-031-37703-7_10

[6] Blicha, M., Fedyukovich, G., Hyvärinen, A.E.J., Sharygina, N.: Transition power abstractions for deep counterexample detection. In: Fisman, D., Rosu, G. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022. Lecture Notes in Computer Science, vol. 13243, pp. 524–542. Springer (2022), https://doi.org/10.1007/978-3-030-99524-9_29

[7] Bradley, A.R., Manna, Z.: Property-directed incremental invariant generation. Formal Aspects Comput. **20**(4-5), 379–405 (2008). https://doi.org/10.1007/S00165-008-0080-9, https://doi.org/10.1007/s00165-008-0080-9

[8] Braverman, M.: Termination of integer linear programs. In: Ball, T., Jones, R.B. (eds.) Computer Aided Verification, 18th International Conference, CAV 2006, Seattle, WA, USA, August 17-20, 2006, Proceedings. pp. 372–385. Lecture Notes in Computer Science, Springer (2006). https://doi.org/10.1007/11817963_34, https://doi.org/10.1007/11817963_34

[9] Brockschmidt, M., Cook, B., Ishtiaq, S., Khlaaf, H., Piterman, N.: T2: temporal property verification. In: Chechik, M., Raskin, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings. pp. 387–393. Lecture Notes in Computer Science, Springer (2016). https://doi.org/10.1007/978-3-662-49674-9_22, https://doi.org/10.1007/978-3-662-49674-9_22

[10] Brockschmidt, M., Ströder, T., Otto, C., Giesl, J.: Automated detection of non-termination and nullpointerexceptions for java bytecode. In: Beckert, B., Damiani, F., Gurov, D. (eds.) Formal Verification of Object-Oriented Software - International Conference, FoVeOOS 2011, Turin, Italy, October 5-7, 2011, Revised Selected Papers. pp. 123–141. Lecture Notes in Computer Science, Springer (2011). https://doi.org/10.1007/978-3-642-31762-0_9, https://doi.org/10.1007/978-3-642-31762-0_9

[11] Chen, H.Y., Cook, B., Fuhs, C., Nimkar, K., O'Hearn, P.W.: Proving nontermination via safety. In: Ábrahám, E., Havelund, K. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings. pp. 156–171. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-642-54862-8_11, https://doi.org/10.1007/978-3-642-54862-8_11

[12] Cook, B., Podelski, A., Rybalchenko, A.: Termination proofs for systems code. In: Schwartzbach, M.I., Ball, T. (eds.) Proceedings of the ACM SIGPLAN 2006 Conference on Programming Language Design and Implementation, Ottawa, Ontario, Canada, June 11-14, 2006. pp. 415–426. ACM (2006). https://doi.org/10.1145/1133981.1134029, https://doi.org/10.1145/1133981.1134029

[13] Craig, W.: Three uses of the Herbrand-Gentzen theorem in relating model theory and proof theory. The Journal of Symbolic Logic **22**(3), 269–285 (1957)

[14] Frohn, F., Giesl, J.: Proving non-termination and lower runtime bounds with loat (system description). In: Blanchette, J., Kovács, L., Pattinson, D. (eds.) Automated Reasoning - 11th International Joint Conference, IJCAR 2022, Haifa, Israel, August 8-10, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13385, pp. 712–722. Springer (2022). https://doi.org/10.1007/978-3-031-10769-6_41, https://doi.org/10.1007/978-3-031-10769-6_41

[15] Frohn, F., Hark, M., Giesl, J.: Termination of polynomial loops. In: Pichardie, D., Sighireanu, M. (eds.) Static Analysis - 27th International Symposium, SAS 2020, Virtual Event, November 18-20, 2020, Proceedings. pp. 89–112. Lecture Notes in Computer Science, Springer (2020). https://doi.org/10.1007/978-3-030-65474-0_5, https://doi.org/10.1007/978-3-030-65474-0_5

[16] Giesl, J., Aschermann, C., Brockschmidt, M., Emmes, F., Frohn, F., Fuhs, C., Hensel, J., Otto, C., Plücker, M., Schneider-Kamp, P., Ströder, T., Swiderski, S., Thiemann, R.: Analyzing program termination and complexity automatically with aprove. J. Autom. Reason. **58**(1), 3–31 (2017). https://doi.org/10.1007/S10817-016-9388-Y, https://doi.org/10.1007/s10817-016-9388-y

[17] Gupta, A., Henzinger, T.A., Majumdar, R., Rybalchenko, A., Xu, R.: Proving non-termination. In: Necula, G.C., Wadler, P. (eds.) Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008. pp. 147–158. ACM (2008). https://doi.org/10.1145/1328438.1328459, https://doi.org/10.1145/1328438.1328459

[18] Hark, M., Frohn, F., Giesl, J.: Termination of triangular polynomial loops. Formal Methods Syst. Des. **65**(1), 70–132 (2025). https://doi.org/10.1007/S10703-023-00440-Z, https://doi.org/10.1007/s10703-023-00440-z

[19] Heizmann, M., Hoenicke, J., Podelski, A.: Termination analysis by learning terminating programs. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. pp. 797–813. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_53, https://doi.org/10.1007/978-3-319-08867-9_53

[20] Hyvärinen, A.E.J., Marescotti, M., Alt, L., Sharygina, N.: Opensmt2: An SMT solver for multi-core and cloud computing. In: Creignou, N., Berre, D.L. (eds.) Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference, Bordeaux, France, July 5-8, 2016, Proceedings. Lecture Notes in Computer Science, vol. 9710, pp. 547–553. Springer (2016). https://doi.org/10.1007/978-3-319-40970-2_35, https://doi.org/10.1007/978-3-319-40970-2_35

[21] Kroening, D., Sharygina, N., Tsitovich, A., Wintersteiger, C.M.: Termination analysis with compositional transition invariants. In: Touili, T., Cook, B., Jackson, P.B. (eds.) Computer Aided Verification, 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings. pp. 89–103. Lecture Notes in Computer Science, Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_9, https://doi.org/10.1007/978-3-642-14295-6_9

[22] Kura, S., Unno, H., Hasuo, I.: Decision tree learning in cegis-based termination analysis. In: Silva, A., Leino, K.R.M. (eds.) Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II. pp. 75–98. Lecture Notes in Computer Science, Springer (2021). https://doi.org/10.1007/978-3-030-81688-9_4, https://doi.org/10.1007/978-3-030-81688-9_4

[23] Larraz, D., Nimkar, K., Oliveras, A., Rodríguez-Carbonell, E., Rubio, A.: Proving non-termination using max-smt. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. pp. 779–796. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_52, https://doi.org/10.1007/978-3-319-08867-9_52

[24] Le, T.C., Qin, S., Chin, W.: Termination and non-termination specification inference. In: Grove, D., Blackburn, S.M. (eds.) Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015. pp. 489–498. ACM (2015). https://doi.org/10.1145/2737924.2737993, https://doi.org/10.1145/2737924.2737993

[25] Leike, J., Heizmann, M.: Geometric nontermination arguments. In: Beyer, D., Huisman, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 24th International Conference, TACAS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings, Part II. pp. 266–283. Lecture Notes in Computer Science, Springer (2018). https://doi.org/10.1007/978-3-319-89963-3_16, https://doi.org/10.1007/978-3-319-89963-3_16

[26] Lommen, N., Giesl, J.: Aprove (koat + loat). In: Gurfinkel, A., Heule, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 205–211. Springer Nature Switzerland, Cham (2025)

[27] McMillan, K.L.: Interpolation and sat-based model checking. In: Jr., W.A.H., Somenzi, F. (eds.) Computer Aided Verification, 15th International Conference, CAV 2003, Boulder, CO, USA, July 8-12, 2003, Proceedings. Lecture Notes in Computer Science, vol. 2725, pp. 1–13. Springer (2003), https://doi.org/10.1007/978-3-540-45069-6_1

[28] Podelski, A., Rybalchenko, A.: A complete method for the synthesis of linear ranking functions. In: Steffen, B., Levi, G. (eds.) Verification, Model Checking, and Abstract Interpretation, 5th International Conference, VMCAI 2004, Venice, Italy, January 11-13, 2004, Proceedings. pp. 239–251. Lecture Notes in Computer Science, Springer (2004). https://doi.org/10.1007/978-3-540-24622-0_20, https://doi.org/10.1007/978-3-540-24622-0_20

[29] Podelski, A., Rybalchenko, A.: Transition invariants. In: 19th IEEE Symposium on Logic in Computer Science (LICS 2004), 14-17 July 2004, Turku, Finland, Proceedings. pp. 32–41. IEEE Computer Society (2004). https://doi.org/10.1109/LICS.2004.1319598, https://doi.org/10.1109/LICS.2004.1319598

[30] Sarita, Y., Singh, A., Gomber, S., Singh, G., Vishwanathan, M.: Efficient ranking function-based termination analysis via bidirectional decompositional search. In: Krebbers, R. (ed.) Programming Languages and Systems - 35th European Symposium on Programming, ESOP 2026, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2026, Turin, Italy, April 11-16, 2026, Proceedings, Part II. pp. 181–209. Lecture Notes in Computer Science, Springer (2026). https://doi.org/10.1007/978-3-032-22723-2_7, https://doi.org/10.1007/978-3-032-22723-2_7

[31] Tsitovich, A., Sharygina, N., Wintersteiger, C.M., Kroening, D.: Loop summarization and termination analysis. In: Abdulla, P.A., Leino, K.R.M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 17th International Conference, TACAS 2011, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2011, Saarbrücken, Germany, March 26-April 3, 2011. Proceedings. pp. 81–95. Lecture Notes in Computer Science, Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_9, https://doi.org/10.1007/978-3-642-19835-9_9

[32] Unno, H., Terauchi, T., Gu, Y., Koskinen, E.: Modular primal-dual fixpoint logic solving for temporal verification. Proc. ACM Program. Lang. **7**(POPL), 2111–2140 (2023). https://doi.org/10.1145/3571265, https://doi.org/10.1145/3571265

[33] Urban, C., Gurfinkel, A., Kahsai, T.: Synthesizing ranking functions from bits and pieces. In: Chechik, M., Raskin, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings. pp. 54–70. Lecture Notes in Computer Science, Springer (2016). https://doi.org/10.1007/978-3-662-49674-9_4, https://doi.org/10.1007/978-3-662-49674-9_4